AIML 231 — Week 6

Feature Construction

Dr Qi Chen

School of Engineering and Computer Science

Victoria University of Wellington

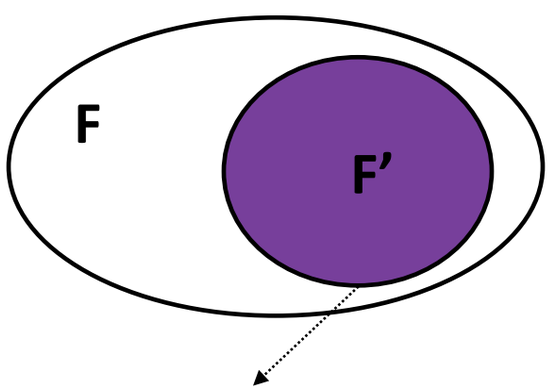
Qi.Chen@vuw.ac.nz

Feature Selection vs. Feature Construction (Recap)

Feature selection (FS)

- Select a subset of relevant features to achieve similar or better performance than using all features

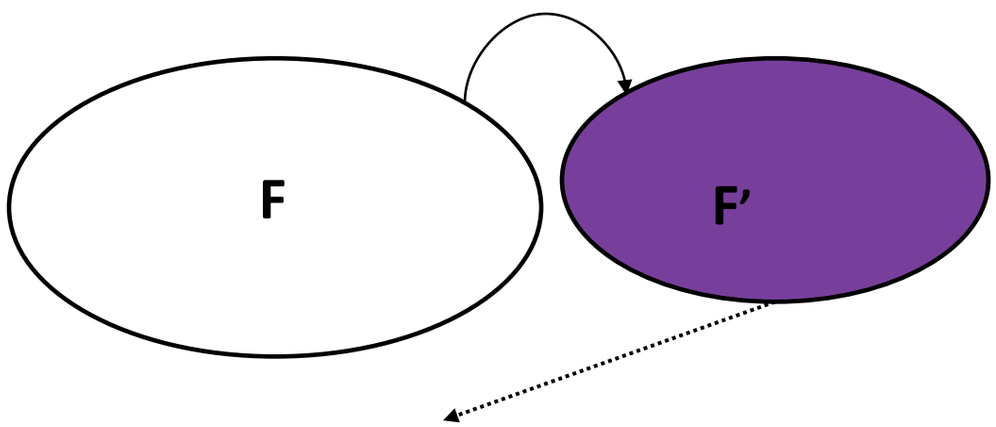
$\mathbf{F} = \{f_1, f_2, \ldots, f_n\}$



$\mathbf{F'} = \{f_{i1}, f_{i2}, \ldots, f_{im}\}$ $(m<n)$

Feature construction (FC)

- Build new high-level features using the original features to improve the classification performance



$\mathbf{F'} = \{g_1(f_1,\ldots, f_n), g_2(f_1, \ldots, f_n), \ldots, g_m(f_1, \ldots, f_n)\}$

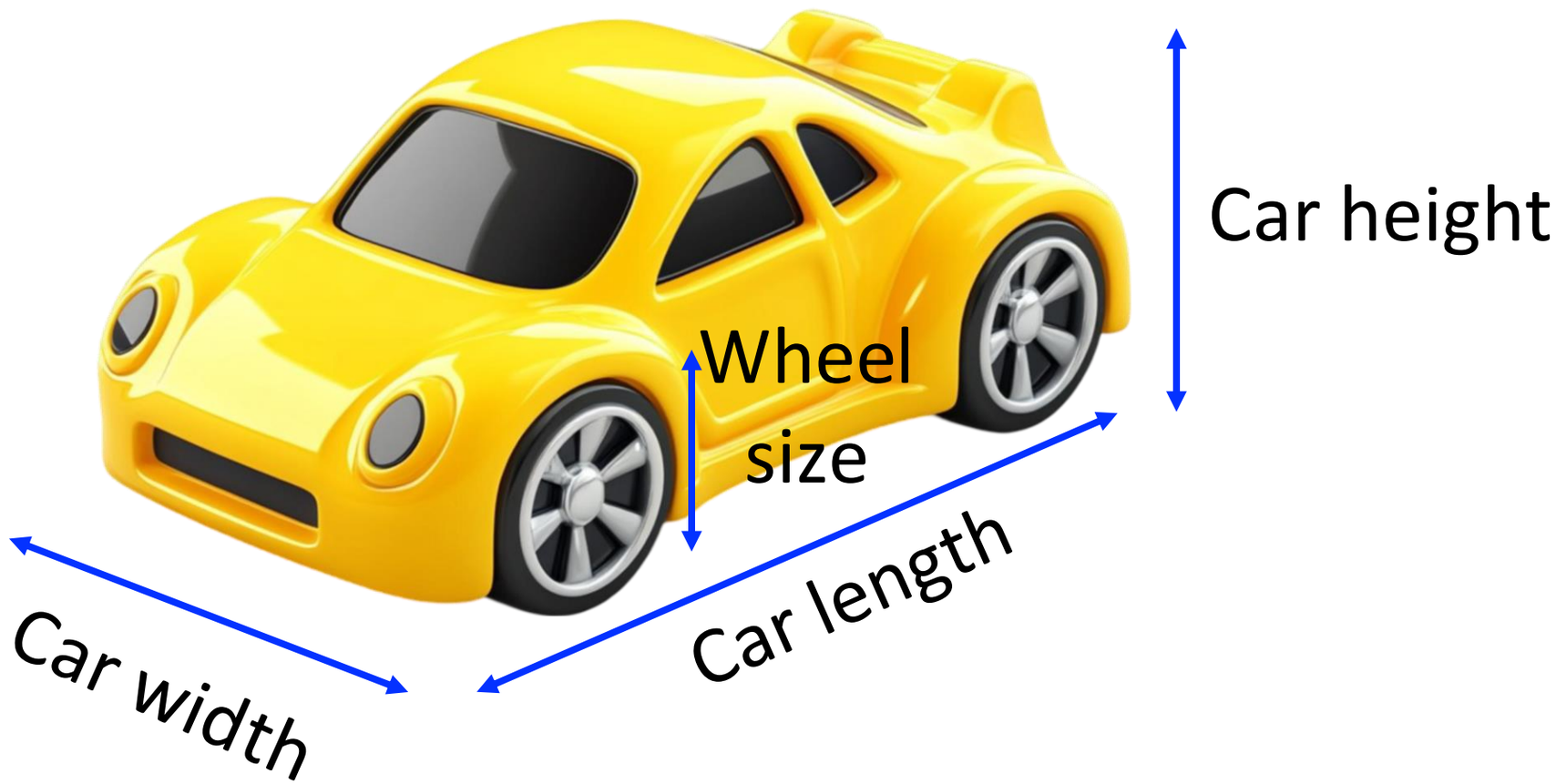
Feature Construction

- A kind of feature transformation to produce high-level constructed features that discover the relationships between features and augment the feature space
- Given $(X_1, X_2, \ldots, X_m)$ - the vector corresponding to the set of original features, a constructed feature is a scalar function φ that transforms the set to a one-dimensional space: $\varphi(X_1, X_2, \ldots, X_m)$

Example: Give $[X_1, X_2, X_3]$,
linear construction: $Xc=X_1+X_2$, $Xc=4X_1+3X_2+6X_3$, …
nonlinear construction: $Xc=X_1*X_2$, $Xc=X_2*X_3^2$, …
…

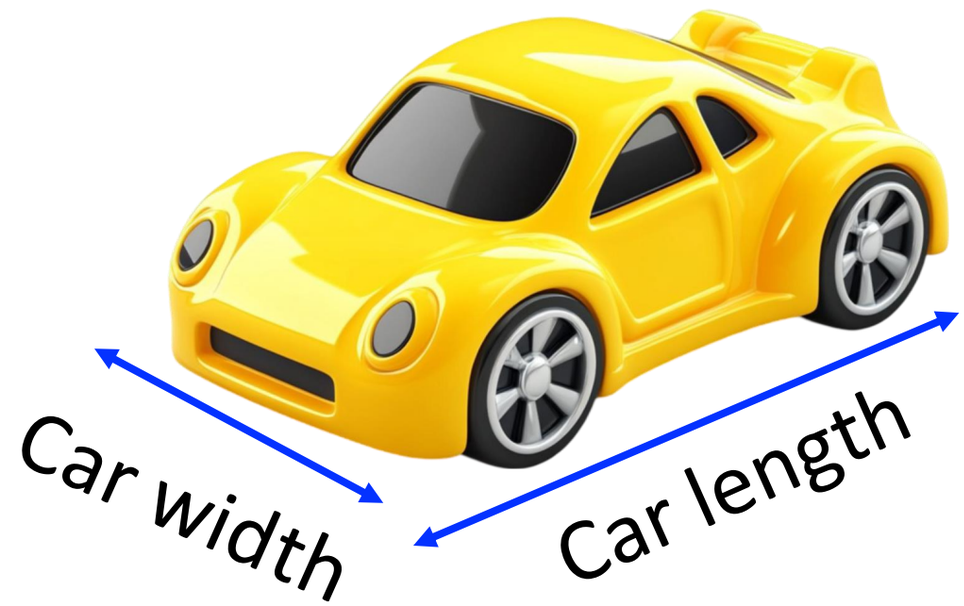
- We will focus on:
  - PCA
  - Kernel PCA
  - Polynomial Features

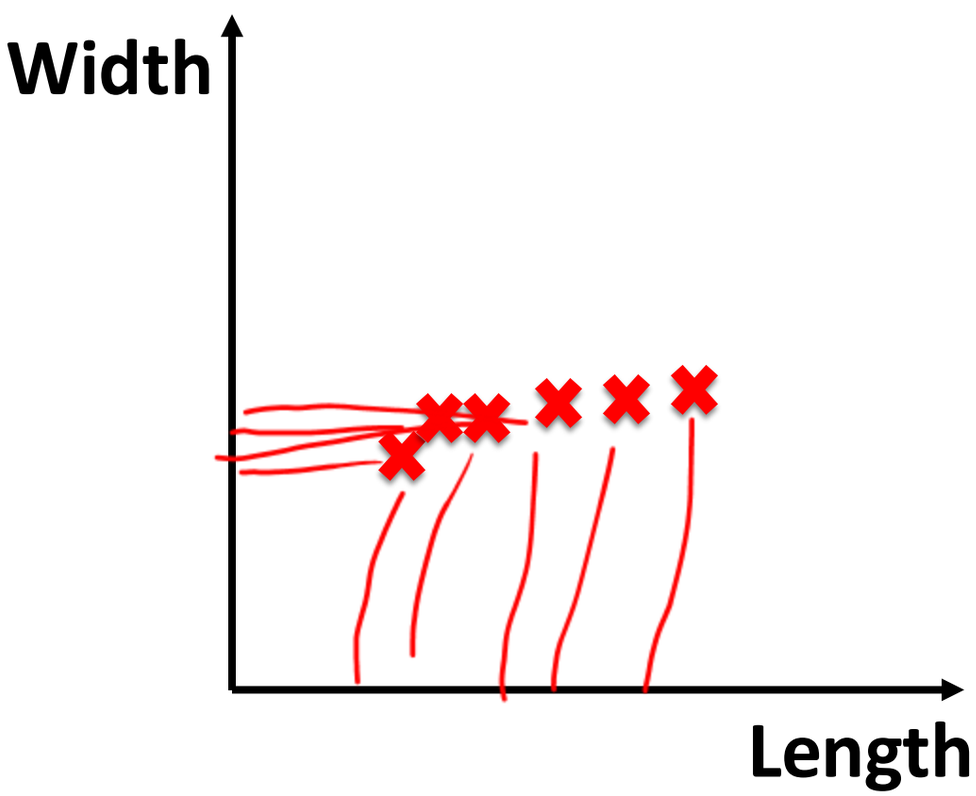
Example: Car Dataset (1)



Example: Car Dataset (2)

Car Width	Car Length
1.7	4
…	…
1.75	4.9

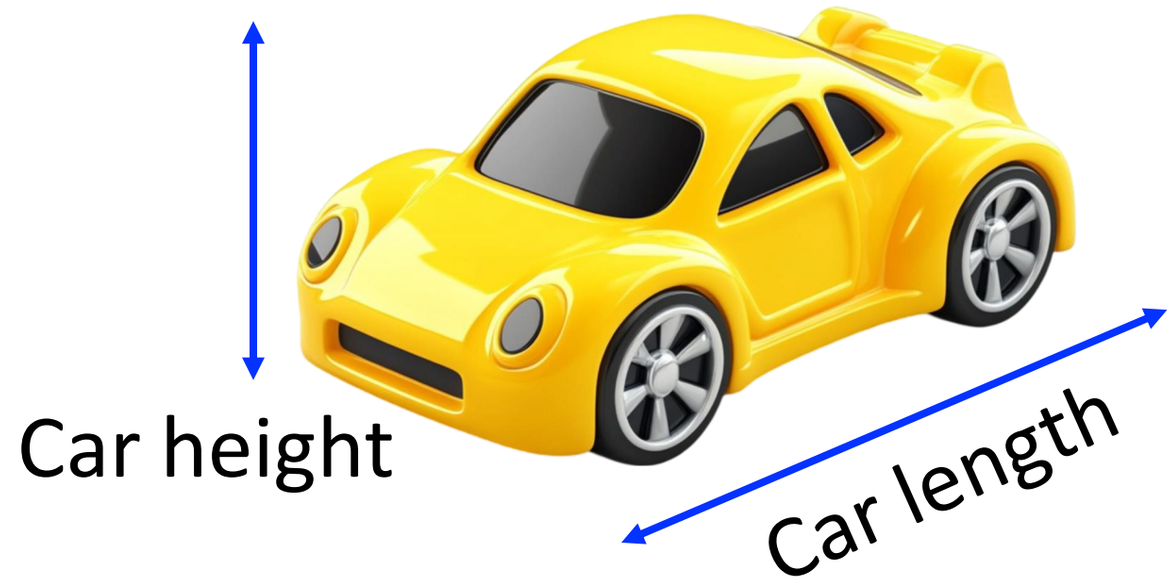


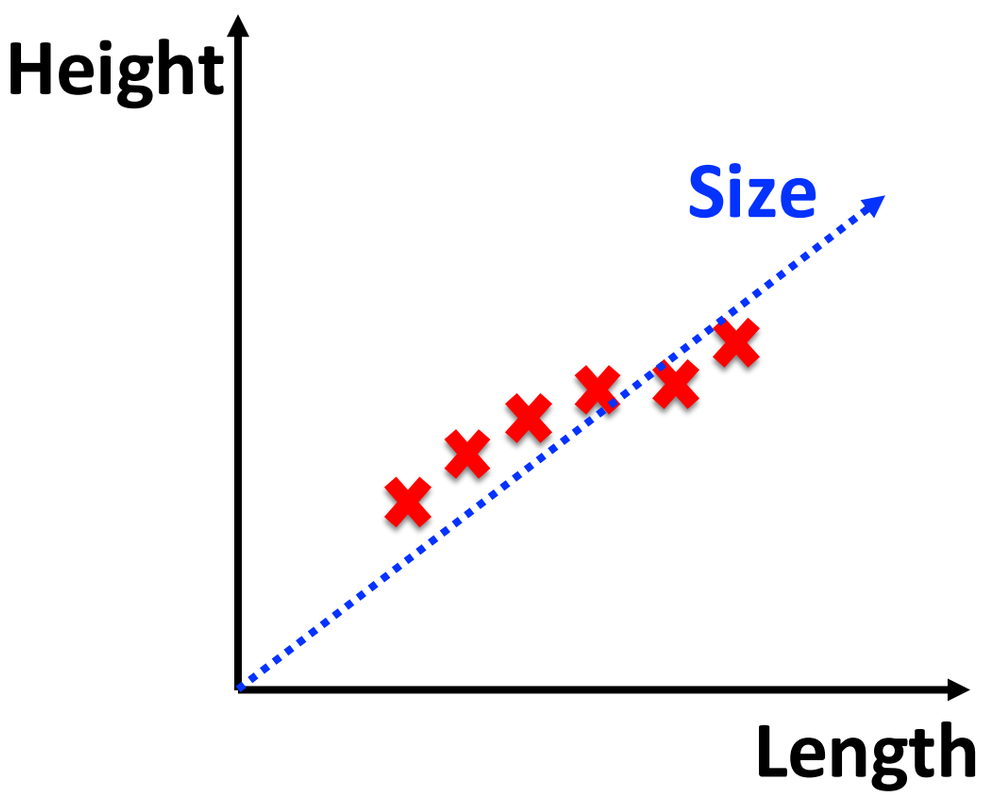


- Car width is usually *constrained* by the road (lane) width, so it varies *little*.
  - Low variance.
- Car length varies more.
  - High variance.
- Use only 1 feature, which one?

Example: Car Dataset (2)

Car Height	Car Length
1.4	3
…	…
1.8	7

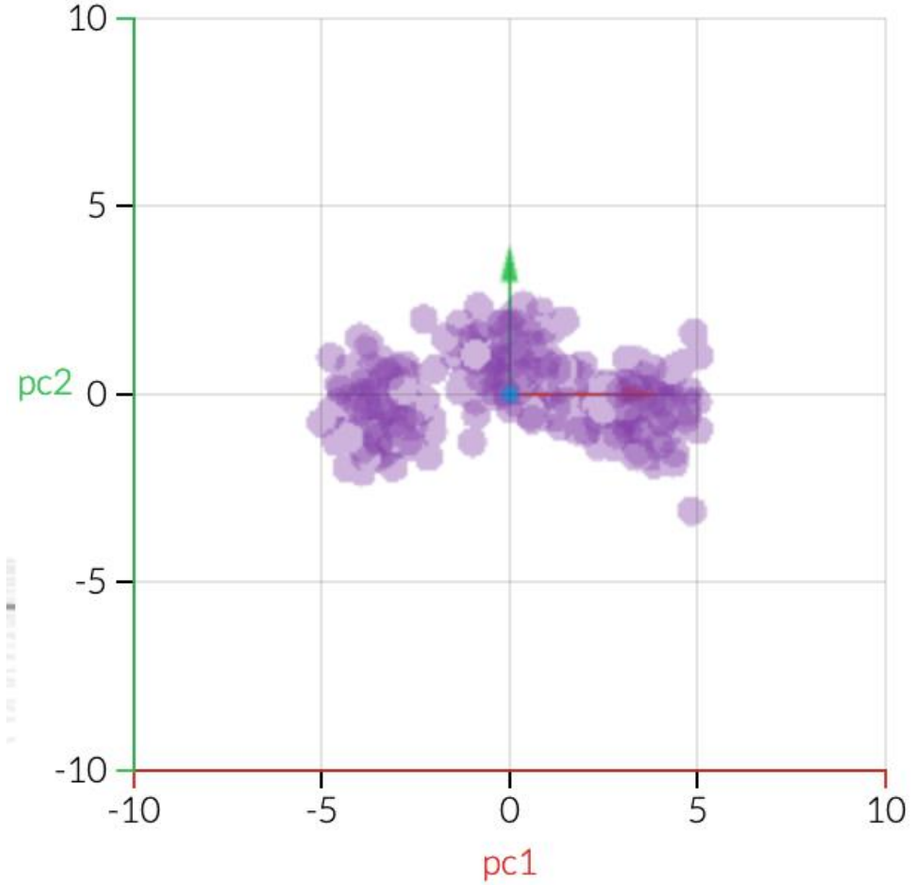


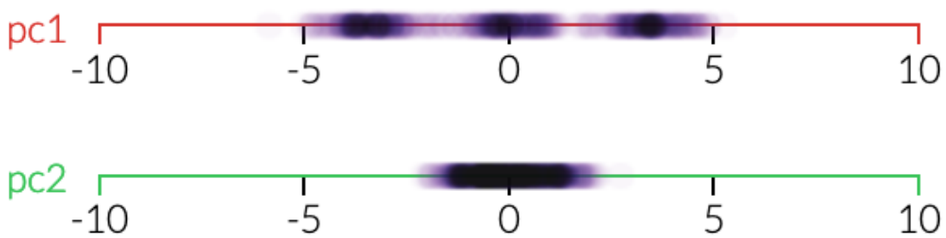


- Both car height and car length have high variance.
- Use only 1 feature, which one?
  - Cannot pick either of height or length due to *information loss*.
  - can build a new feature Size that combines *Height* and *Length linearly* and contains information in both *Height* and *Length.*

Principal Component Analysis (PCA)

- Invented by Karl Pearson (1901)
- *PCA* is a mathematical procedure that ***linearly*** transforms *(possibly) correlated* features into a (*smaller*) number of *uncorrelated (independent)* features called *principal components*.
- Goal is to achieve *high data variance.*





- Data variance is related to the distances between data points.
- Higher variance: easier to extract knowledge (for example building a classifier to separate different classes).
- pc1 has higher variance than pc2.

Principal Components

From d original features: $x_1, x_2, \ldots, x_d$:

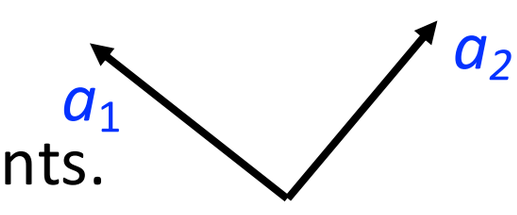
Produce p new features: $z_1, z_2, \ldots, z_p$:

$$\left.\begin{aligned} z_1 &= a_{11}x_1 + a_{12}x_2 + \ldots + a_{1d}x_d \\ z_2 &= a_{21}x_1 + a_{22}x_2 + \ldots + a_{2d}x_d \\ &\ldots \\ z_p &= a_{p1}x_1 + a_{p2}x_2 + \ldots + a_{pd}x_d \end{aligned}\right\}$$

z_p's are Principal Components

z_p's are independent

- Each principal component is represented by a weight vector.
  - E.g., z_1 is represented by the weight vector $a_1 = (a_{11}, a_{12}, \ldots, a_{1d})$.
- The weight vectors of two principal components are orthogonal.
  - $a_1 \cdot a_2 = a_{11}a_{21} + a_{12}a_{22} + \ldots + a_{1d}a_{2d} = 0$.
  - No redundancy exists between any pair of principal components.



- How to find the principal components (weight vectors)?

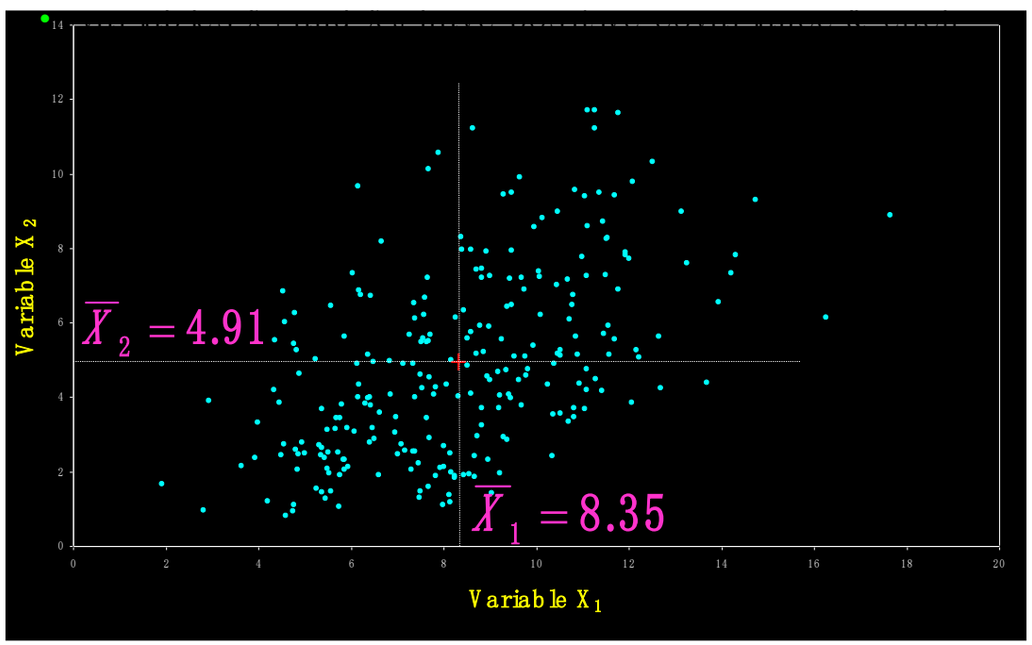
PCA Algorithm

Build based on the training set only!

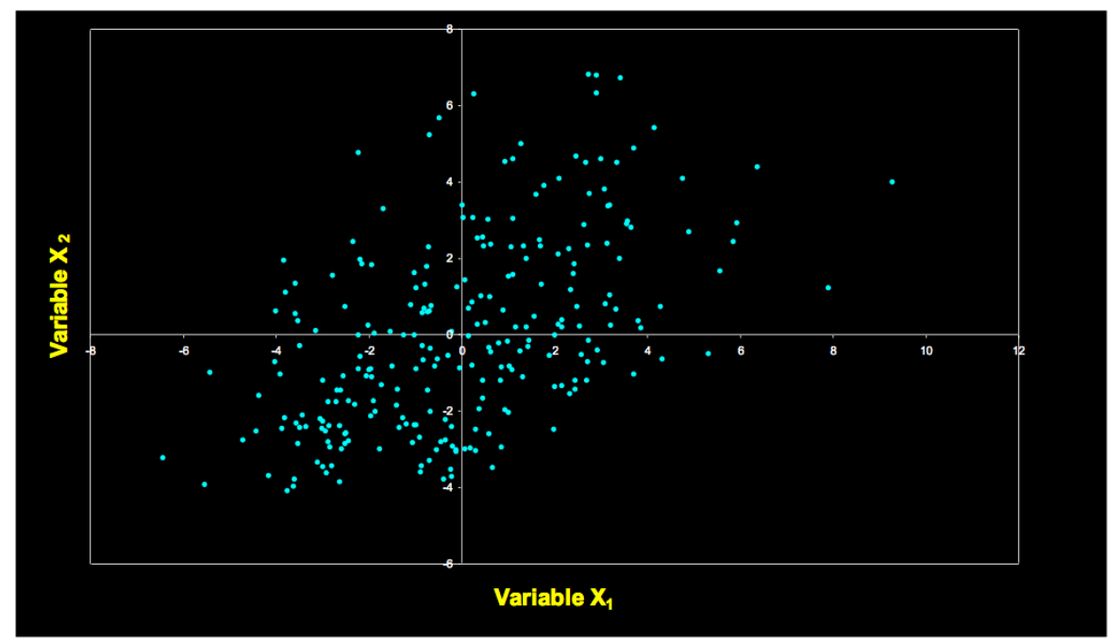
1. Centralising the data:
   - Ensure each feature has mean of 0 by subtracting mean from each feature.

$$x_i = x_i - \mu_i$$

where μ_i is the mean of the *i*th feature.



Centralising



PCA Algorithm

1. Centralising the data.
2. (Optional) Scaling the data.
   - Ensure all features have the same range.
   - Why?

PCA Algorithm

1. Centralising the data.
2. (Optional) Scaling the data.
3. Compute the covariance matrix **C** (*d* x *d*) where *d* is the number of features and *n* is the number of instances.
   - C_{ij} is the covariance between features x_i and x_j.
   - C_{ii} is the variance of features x_i.

$$\begin{array}{c|ccc} & x_1 & x_2 & x_3 \\ \hline x_1 & 2.0 & 0.8 & -0.5 \\ x_2 & 0.8 & 0.1 & 0.2 \\ x_3 & -0.5 & 0.2 & 1.1 \end{array}$$

$$cov(x_i, x_j) = \frac{1}{n} \sum_{k=1}^{n} x_{ik} \times x_{jk}$$

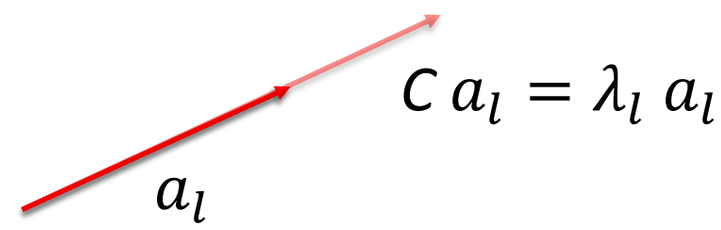
$$var(x_i) = \frac{1}{n} \sum_{k=1}^{n} {x_{ik}}^2$$

- **C** is symmetric as $cov(x_i, x_j) = cov(x_j, x_i)$

PCA Algorithm

1. Centralising the data.
2. (Optional) Scaling the data.
3. Compute the covariance matrix **C** (*d* x *d*) where *d* is the number of features.
4. Perform eigen decomposition on the covariance matrix **C**.
   - Compute eigenvalues λ_l and eigenvectors a_l:

$$C\, a_l = \lambda_l\, a_l$$



   - **C** has *d* eigenvectors and *d* corresponding eigenvalues.
   - All eigenvectors are pairwise orthogonal.
   - Each eigenvector represents the *weight vector* of a principal component
   - The corresponding eigenvalue represents the *variance* captured by the principal component.
   - With *d* original features, we can build at most *d* principal components.
   - What if we want only *p (<< d)* components?

Build *p (<< d)* components?

- Calculate the explained variance ratio of each principal component z_l

$$ExpVar(z_l) = \frac{\lambda_l}{\sum_{k=1}^{d} \lambda_k}$$

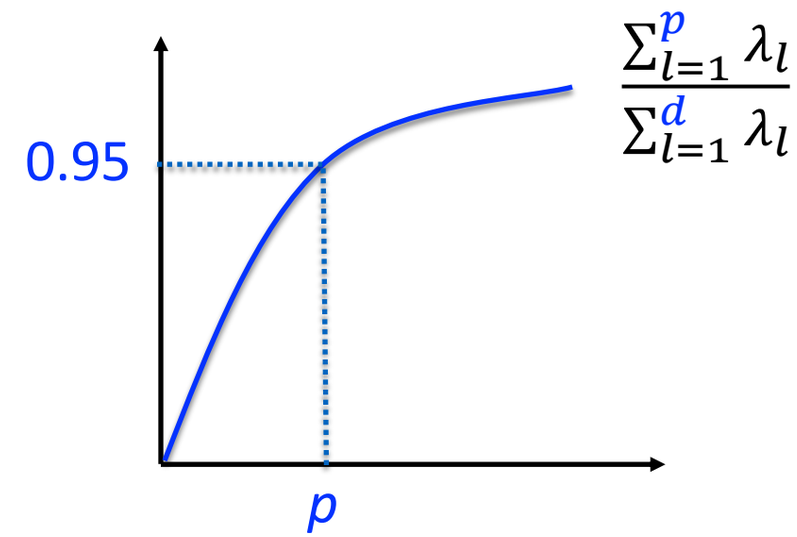
  - $ExpVar(z_l) \in [0, 1]$: *how much variance* that z_l preserves from the original data.
  - The *higher* the explained variance ratio, the *better* the principal component.

- Rank principal components based on their explained variance ration and select top-ranked *p* components.

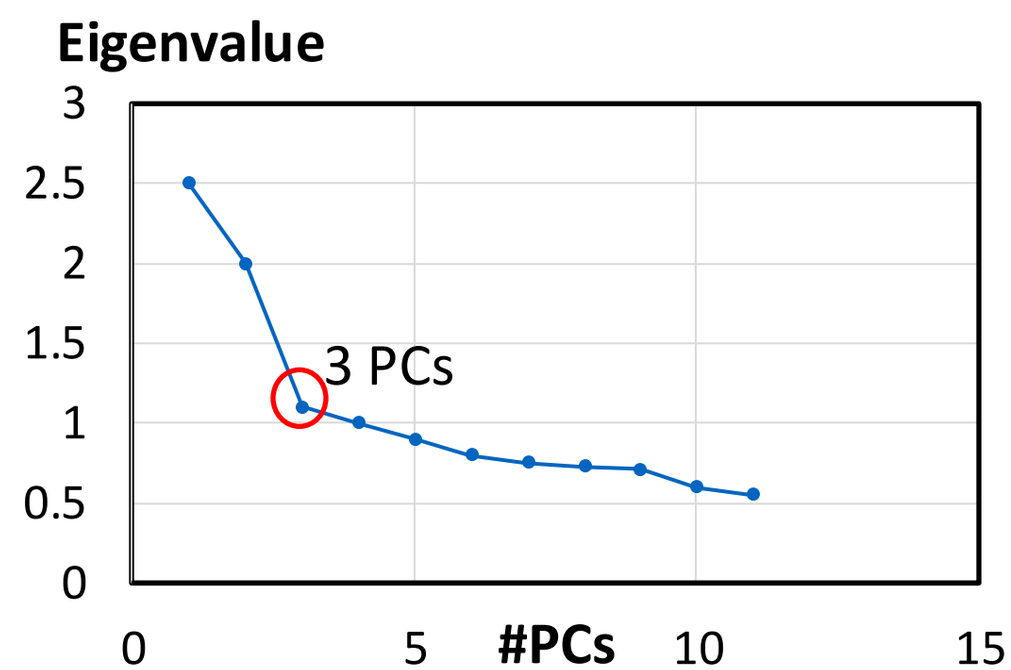
- How to determine *p* ?

How many principal components?

- Cumulative Explained Variance:
  - Pick first p eigenvectors which explain 95% of the total variance.



- Use scree plot:
  - Rank principal components by their eigenvalues.
  - Create a scatter of eigenvalues.
  - Select the knee point, where the explained variance starts to level off.



PCA Discussion

- Is PCA a supervised on unsupervised algorithm?

- Advantages:
  - Ensure new features (principal components) are uncorrelated.
  - Interpretability: the new features are easy to understand.
  - Efficiency: fast to run and scale well with large datasets.

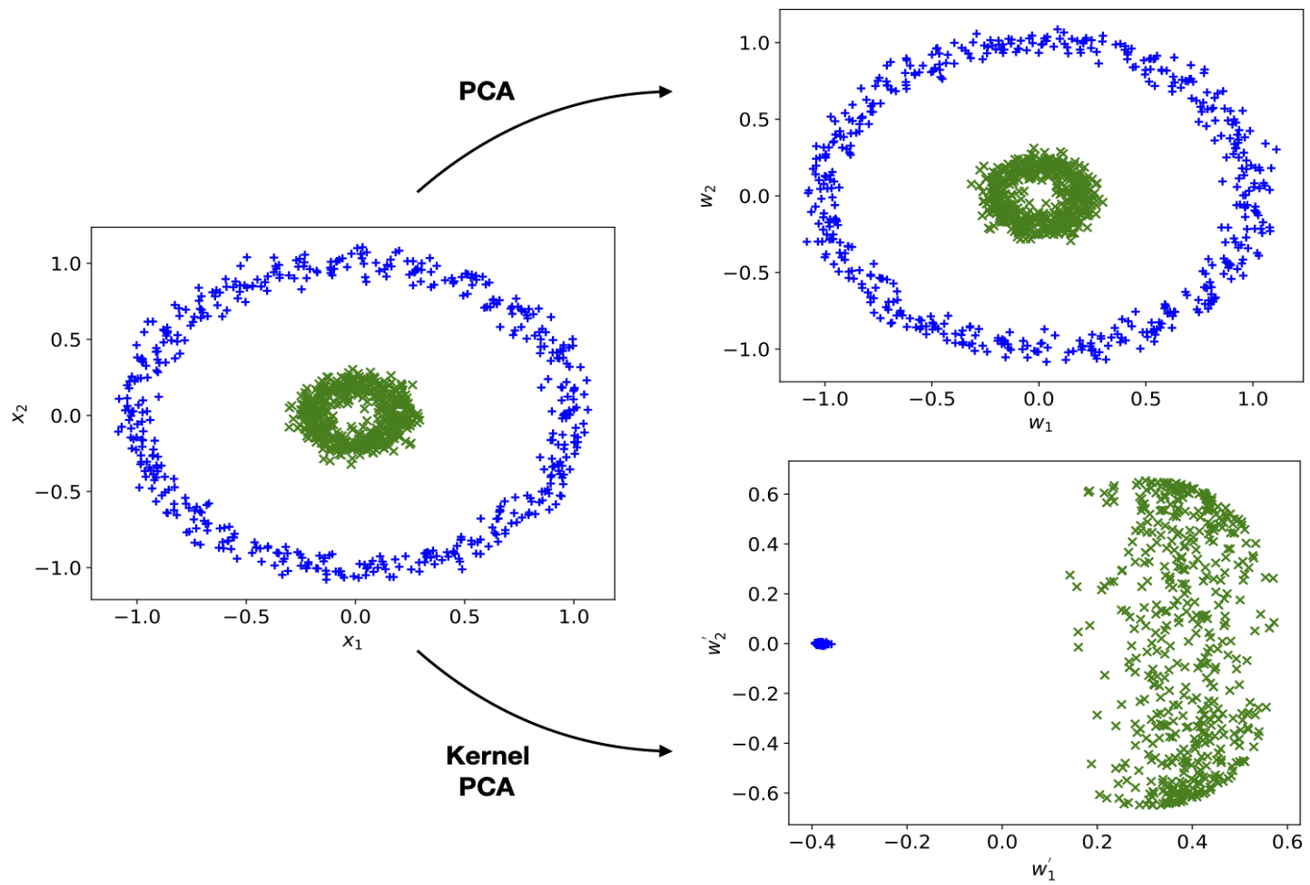
- Limitations:
  - Assume a linear mapping from the original features to the new features.
  - Need to define the number of new principal components.
  - Sometimes uncorrelated features are not sufficient.

- Use *sklearn.decomposition.PCA.*

PCA code example

```python
# Importing necessary libraries
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Load the Iris dataset
iris = load_iris()
X = iris.data  # Features (sepal length, sepal width, petal length, petal width)
y = iris.target  # Target (species)

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Standardize the data (scaling it to have mean=0 and std=1)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Apply PCA to the training set (reduce to 2 components)
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

Kernel Principal Component Analysis

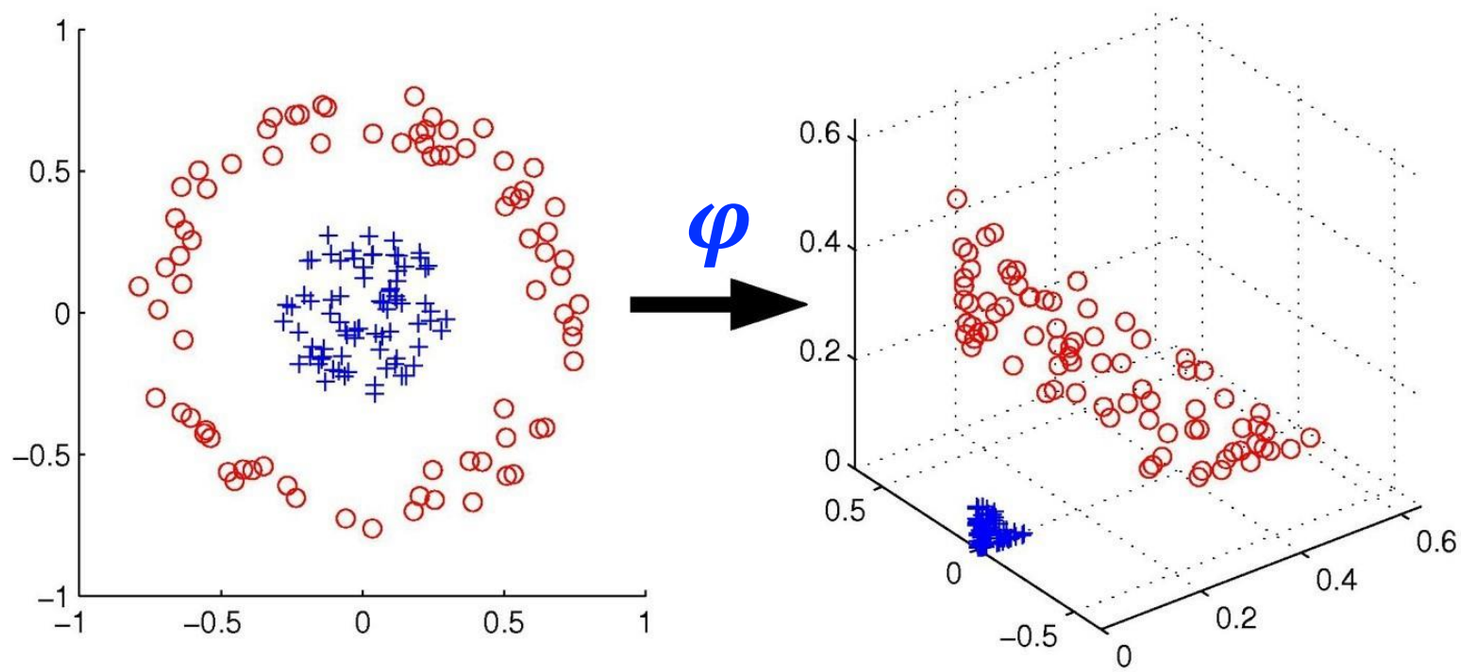


https://ml-lectures.org/docs/structuring_data/ml_without_neural_network-2.html

- Scenario: Two clear “green” and “blue” clusters, but they are not linearly separable.
- PCA does not work.
- Solution: Kernel PCA.

Intuition of Kernel PCA

- Mapping to a higher dimensional space
  - Transform non-linear low-dimensional space to a linear-separable higher-dimensional space.
  - Use a function φ to perform the transformation.



- PCA in High-Dimensional Space:
  - Once the data is mapped to the higher-dimensional and linear-separable space, perform Principal Component Analysis (PCA) as usual.

Kernel Trick

- The challenge: How to find $\boldsymbol{\varphi}$?
  - Explicitly computing $\boldsymbol{\varphi}$ is difficult and computationally expensive.

- The solution: We do not need to compute $\boldsymbol{\varphi}$ explicitly:
  - In kernel PCA, we compute similarities between a pair of transformed instances: $cov\big(\boldsymbol{\varphi}(x_i), \boldsymbol{\varphi}(x_j)\big) = K(x_i, x_j)$ where *K()* is a kernel function.

- Kernel trick: $\boldsymbol{\varphi}$ is embedded in *K().*

- Common kernel choices:
  - Polynomial kernel: $K\big(x_i, x_j\big) = (x_i \cdot x_j + c)^d$
  - Gaussian (RBF) kernel: $K\big(x_i, x_j\big) = \exp(-\frac{\left\|x_i - x_j\right\|^2}{2\sigma^2})$

Comments on Kernel PCA

- Advantages:
  - Handles non-linear data.
  - Works well on high-dimensional data.

- Limitations:
  - Implicit mapping hidden behinds the kernel function -> loss of interpretability.
  - Choosing the right kernel (including kernel parameters) is tricky.

- *sklearn.decomposition.KernelPCA*

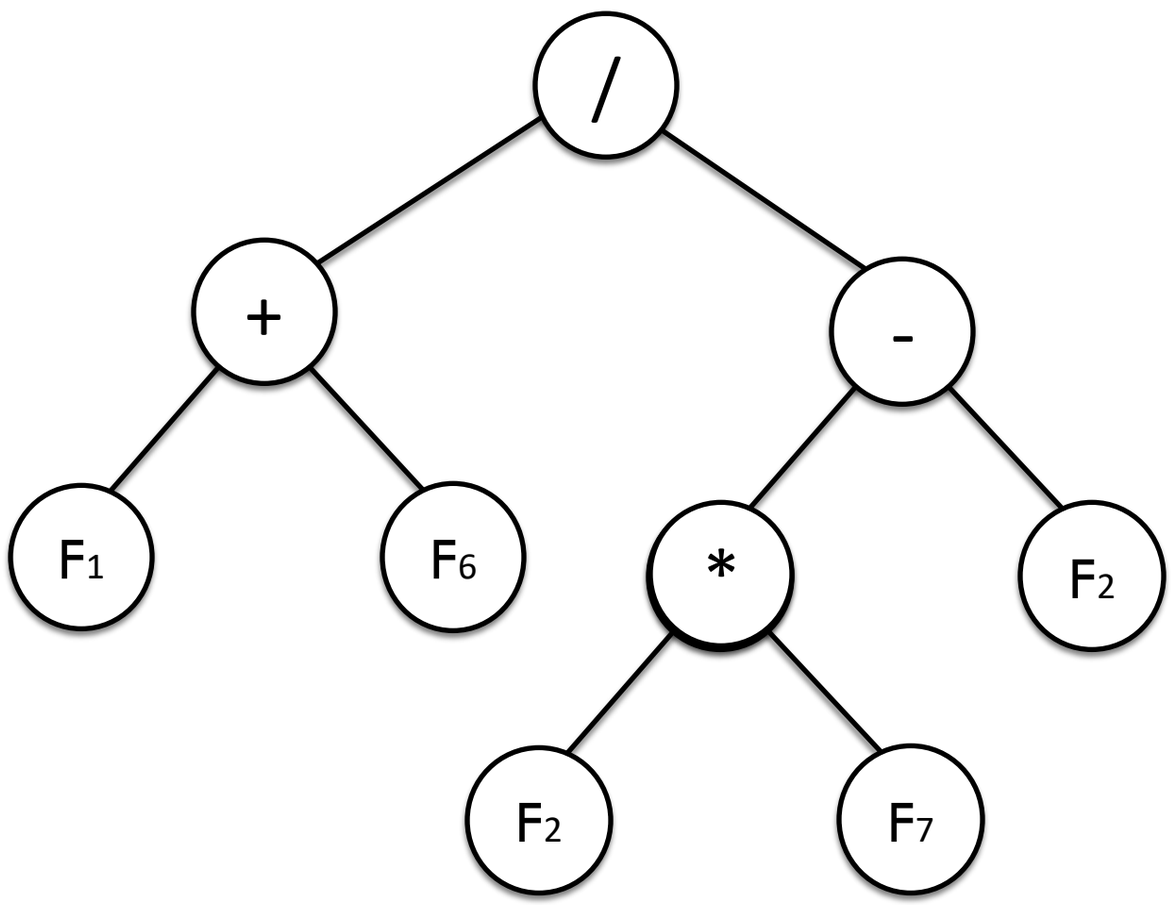
Polynomial Features

- Polynomial features are created by raising existing features to an exponent.
- Generate a new feature matrix with the polynomial combinations of the features with a degree (less than or equal to the specified degree).

Example: Two-dimensional input feature space $[x_1, x_2]$
the ***degree-2*** polynomial features: $[1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$

- Higher degree leads to larger number of features -> consider feature selection
- Polynomial features separate small and large values, changing the probability distribution of the features.
- Appropriate degree can improve ML algorithms, typically linear algorithms.

- *sklearn.preprocessing.PolynomialFeatures*

GP For Feature Construction (Bonus)

- **Genetic Programming** is flexible in making mathematical and logical functions
- There isn't much structural (topological) information in the search space of possible functions, so using a meta-heuristic approach (such as evolutionary computation) seems reasonable
- More in **AIML427: Big Data** 😁